

(Matching and Subclassification)——

2026-03-23

- (DAG)
-
-
- $do-$

. . .

-
- (PSM)
-
-

(RCT)

-
- $D \perp \{Y(0), Y(1)\}$
-
-

-
-
-
-

...

$$D \not\perp \{Y(0), Y(1)\}$$

$$\underbrace{E[Y|D = 1] - E[Y|D = 0]} = \underbrace{E[Y(1) - Y(0)|D = 1]}_{\text{ATT}} + \underbrace{\{E[Y(0)|D = 1] - E[Y(0)|D = 0]\}}$$

...

-
- ” ”

(Ignorability)

$$X$$

$$D \perp \{Y(0), Y(1)\} | X$$

(Unconfoundedness)

...

- X
- X
- X

...

-
-
- (Overlap)

” ”

- -

vs

(Exact Matching)

-
-
- ” ”

(Approximate Matching)

- ” ”
- ” ” ” ”
-

—
— (Caliper matching)
—

...

- $d(X_i, X_j) = \sqrt{(X_i - X_j)'(X_i - X_j)}$ -

vs

1:1	1
1:k	k

(PSM)

(Propensity Score)

X

$$e(X) = P(D = 1|X)$$

Rosenbaum & Rubin (1983) $D \perp \{Y(0), Y(1)\} | X$ $D \perp \{Y(0), Y(1)\} | e(X)$

...

- X $e(X)$
-
-

...

- Logistic - Probit -

PSM

1

Logit Probit

$$\text{logit}(e(X)) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_k X_k$$

$$\hat{e}(X_i)$$

...

2 (Common Support)

-
-
-

... .

3

-
-

PSM

(Nearest Neighbor)

- /
- 1:1 1:k
-

(Caliper Matching)

- ” ”
- $0.2 \times SD(\hat{e}(X))$
-

... .

(Kernel Matching)

$$\hat{Y}_0(1) = \frac{\sum_{j \in \text{Control}} K(\frac{e_i - e_j}{h}) Y_j}{\sum_{j \in \text{Control}} K(\frac{e_i - e_j}{h})}$$

$$K(\cdot) \quad h$$

(Subclassification)

(strata)

- - " " -
 . . .
 1. 2. J J = 5 10 3. 4.

$$\hat{\tau}_j = \bar{Y}_{1j} - \bar{Y}_{0j}$$

$\bar{Y}_{1j}$ $\bar{Y}_{0j}$ j
 . . .

$$\hat{\tau} = \sum_{j=1}^J \frac{N_j}{N} \hat{\tau}_j$$

N_j j N
 . . .

💡

- Cochran (1968) 5-10
-
-

•
•
•

...
- (Standardized Bias) - QQ

$$\text{StdBias} = \frac{\bar{X}_{\text{treat}} - \bar{X}_{\text{control}}}{\sqrt{(s^2_{\text{treat}} + s^2_{\text{control}})/2}}$$

...
(Rosenbaum & Rubin, 1985) - < 0.1 - 0.1 ≤ < 0.2 - ≥ 0.2

...
- 0.1 -

- -

...
- - -

...

- 1.
2. **QQ**
- 3.
4. **Love**

LaLonde

LaLonde (1986) (NSW)

-
-
- 1978

...

- - -

...

DID

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import NearestNeighbors
import warnings
warnings.filterwarnings('ignore')

#
plt.rcParams['font.sans-serif'] = ['Source Han Sans SC', 'Noto Sans CJK SC',
                                   'WenQuanYi Micro Hei', 'SimHei']
plt.rcParams['axes.unicode_minus'] = False

# LaLonde
np.random.seed(42)
n = 1000

#
age = np.random.normal(25, 7, n)
education = np.random.normal(10, 2.5, n)
```



```

black = np.random.binomial(1, 0.5, n)
hispanic = np.random.binomial(1, 0.2, n)
married = np.random.binomial(1, 0.3, n)
nodegree = np.random.binomial(1, 0.6, n)

#
re74 = np.random.lognormal(8, 1.5, n)
re75 = np.random.lognormal(8, 1.5, n)

#
X_vars = np.column_stack([age, education, black, hispanic, married, nodegree, re74, re75])
X_scaled = (X_vars - X_vars.mean(axis=0)) / X_vars.std(axis=0)

#    Logit
logit_ps = (0.3 * X_scaled[:, 0] - 0.5 * X_scaled[:, 1] + 0.8 * X_scaled[:, 2]
            + 0.3 * X_scaled[:, 5] - 0.2 * X_scaled[:, 6] / 1000)
ps = 1 / (1 + np.exp(-logit_ps))
treat = np.random.binomial(1, ps)

#
true_effect = 1794

#
re78_base = (1000 + 50 * age + 200 * education + 300 * married
             + 0.3 * re74 + 0.3 * re75 + np.random.normal(0, 3000, n))
re78 = re78_base + true_effect * treat

#
data = pd.DataFrame({
    'treat': treat,
    'age': age,
    'education': education,
    'black': black,
    'hispanic': hispanic,
    'married': married,
    'nodegree': nodegree,
    're74': re74,
    're75': re75,
    're78': re78,
    'ps': ps
})

```

```

print(f" : {n}")
print(f" : {treat.sum()} ({treat.mean()*100:.1f}%)")
print(f" : {(1-treat).sum()} ({(1-treat).mean()*100:.1f}%)")
print("\n 5 :")
print(data.head())

```

```

: 1000
: 498 (49.8%)
: 502 (50.2%)

```

```
5 :
```

	treat	age	education	black	hispanic	married	nodegree	\
0	0	28.476999	13.498389	0	0	0	1	
1	0	24.032150	12.311584	0	0	1	1	
2	0	29.533820	10.149076	0	1	1	1	
3	0	35.661209	8.382658	0	0	0	1	
4	1	23.360926	11.745558	1	0	0	0	

	re74	re75	re78	ps
0	2261.494770	3424.001541	7103.120751	0.253744
1	547.154879	285.164672	4072.252489	0.262062
2	4489.587368	3696.493144	5370.954964	0.410819
3	132.693233	11018.011030	12716.050564	0.565160
4	2596.876943	456.323184	4886.686369	0.509255

```

#
covariates = ['age', 'education', 'black', 'hispanic', 'married', 'nodegree', 're74', 're75']

balance_before = []
for var in covariates:
    treated_mean = data[data['treat']==1][var].mean()
    control_mean = data[data['treat']==0][var].mean()
    treated_std = data[data['treat']==1][var].std()
    control_std = data[data['treat']==0][var].std()

    #
    std_bias = (treated_mean - control_mean) / np.sqrt((treated_std**2 + control_std**2) / 2)

```

```

    balance_before.append({
        'Variable': var,
        'Treated_Mean': f"{treated_mean:.2f}",
        'Control_Mean': f"{control_mean:.2f}",
        'Std_Bias': f"{std_bias:.3f}"
    })

balance_df = pd.DataFrame(balance_before)
print("      :")
print("=" * 60)
print(balance_df.to_string(index=False))
print("\n    > 0.2    !")

```

```

:
=====
Variable Treated_Mean Control_Mean Std_Bias
age      26.35      23.93    0.358
education 9.64      10.71   -0.440
black     0.67      0.33    0.718
hispanic  0.18      0.19   -0.032
married   0.31      0.30    0.023
nodegree  0.67      0.53    0.290
re74     7336.78   10678.87  -0.147
re75     8820.42   9890.54  -0.040

> 0.2    !

```

```

# Logistic
X = data[['age', 'education', 'black', 'hispanic', 'married', 'nodegree', 're74', 're75']]
y = data['treat']

#
X_scaled = X.copy()
X_scaled['re74'] = X_scaled['re74'] / 1000
X_scaled['re75'] = X_scaled['re75'] / 1000

# Logistic
logit_model = LogisticRegression(max_iter=1000)

```

```

logit_model.fit(X_scaled, y)

#
data['ps_estimated'] = logit_model.predict_proba(X_scaled)[: , 1]

print("Logistic  :")
coef_df = pd.DataFrame({
    'Variable': X.columns,
    'Coefficient': logit_model.coef_[0]
})
print(coef_df.to_string(index=False))

#
fig, ax = plt.subplots(figsize=(7, 4.5))
ax.hist(data[data['treat']==0]['ps_estimated'], bins=30, alpha=0.5,
        label='Control', color='blue', density=True)
ax.hist(data[data['treat']==1]['ps_estimated'], bins=30, alpha=0.5,
        label='Treated', color='red', density=True)
ax.set_xlabel('Propensity Score', fontsize=11)
ax.set_ylabel('Density', fontsize=11)
ax.set_title('Propensity Score Distribution (Before Matching)', fontsize=12, fontweight='bold')
ax.legend()
ax.axvline(data[data['treat']==0]['ps_estimated'].mean(), color='blue',
           linestyle='--', label='Control Mean')
ax.axvline(data[data['treat']==1]['ps_estimated'].mean(), color='red',
           linestyle='--', label='Treated Mean')
plt.tight_layout()
plt.show()

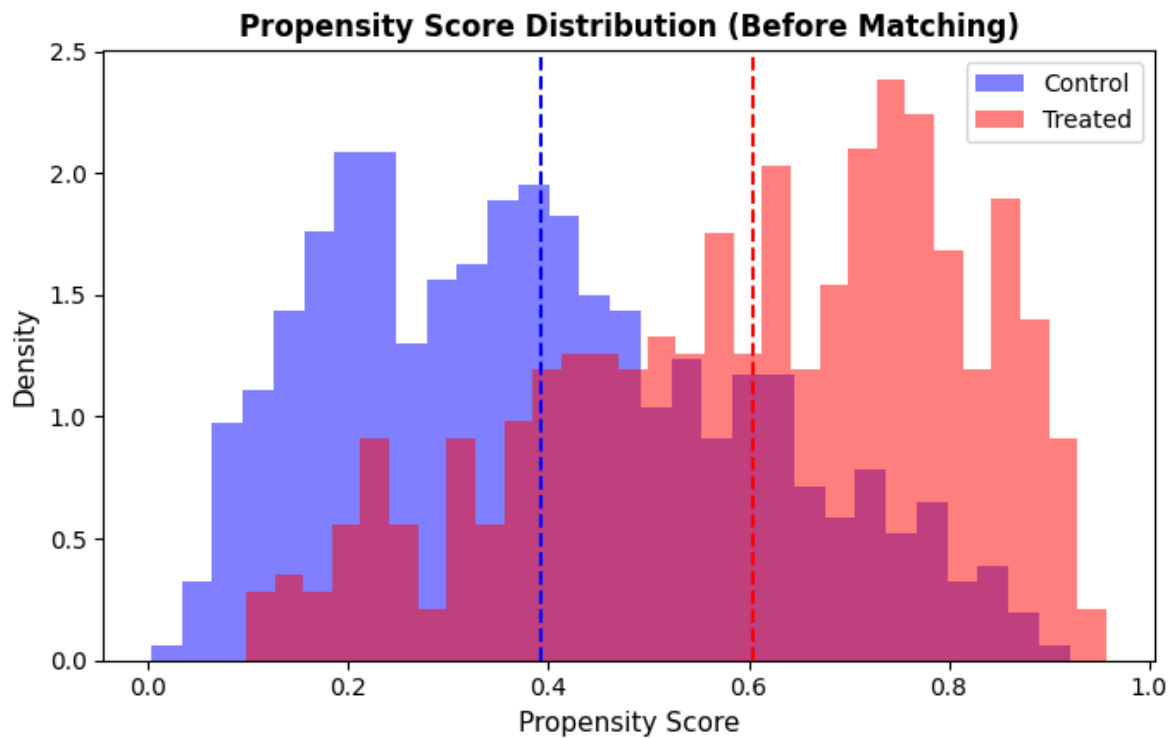
print(f"\n      : {data[data['treat']==1]['ps_estimated'].mean():.3f}")
print(f"      : {data[data['treat']==0]['ps_estimated'].mean():.3f}")

```

```

Logistic  :
Variable  Coefficient
age       0.058077
education -0.216995
black     1.547403
hispanic  -0.060942
married   0.035341
nodegree  0.755832
re74      -0.009717
re75      -0.000560

```



: 0.605
: 0.392

```
#
treated = data[data['treat'] == 1].copy()
control = data[data['treat'] == 0].copy()

# 1:1
nbrs = NearestNeighbors(n_neighbors=1).fit(control['ps_estimated'].values.reshape(-1, 1))
distances, indices = nbrs.kneighbors(treated['ps_estimated'].values.reshape(-1, 1))

#
matched_control_indices = control.iloc[indices.flatten()].index
matched_data = pd.concat([treated, data.loc[matched_control_indices]])
```

```

print(f"      : {len(treated)}")
print(f"      : {len(matched_control_indices)}")
print(f"      : {len(matched_data)}")

#
mean_distance = distances.mean()
print(f"\n      : {mean_distance:.4f}")

```

```

      : 498
      : 498
      : 996

      : 0.0013

```

```

#
balance_after = []
for var in covariates:
    treated_mean = matched_data[matched_data['treat']==1][var].mean()
    control_mean = matched_data[matched_data['treat']==0][var].mean()
    treated_std = matched_data[matched_data['treat']==1][var].std()
    control_std = matched_data[matched_data['treat']==0][var].std()

    std_bias = (treated_mean - control_mean) / np.sqrt((treated_std**2 + control_std**2) / 2)

    balance_after.append({
        'Variable': var,
        'Std_Bias_After': std_bias
    })

#
balance_comparison = pd.DataFrame(balance_before)
balance_comparison['Std_Bias_After'] = [x['Std_Bias_After'] for x in balance_after]
balance_comparison['Std_Bias'] = balance_comparison['Std_Bias'].astype(float)

print("      :")
print("=" * 50)
print(balance_comparison[['Variable', 'Std_Bias', 'Std_Bias_After']].to_string(index=False))

```

```

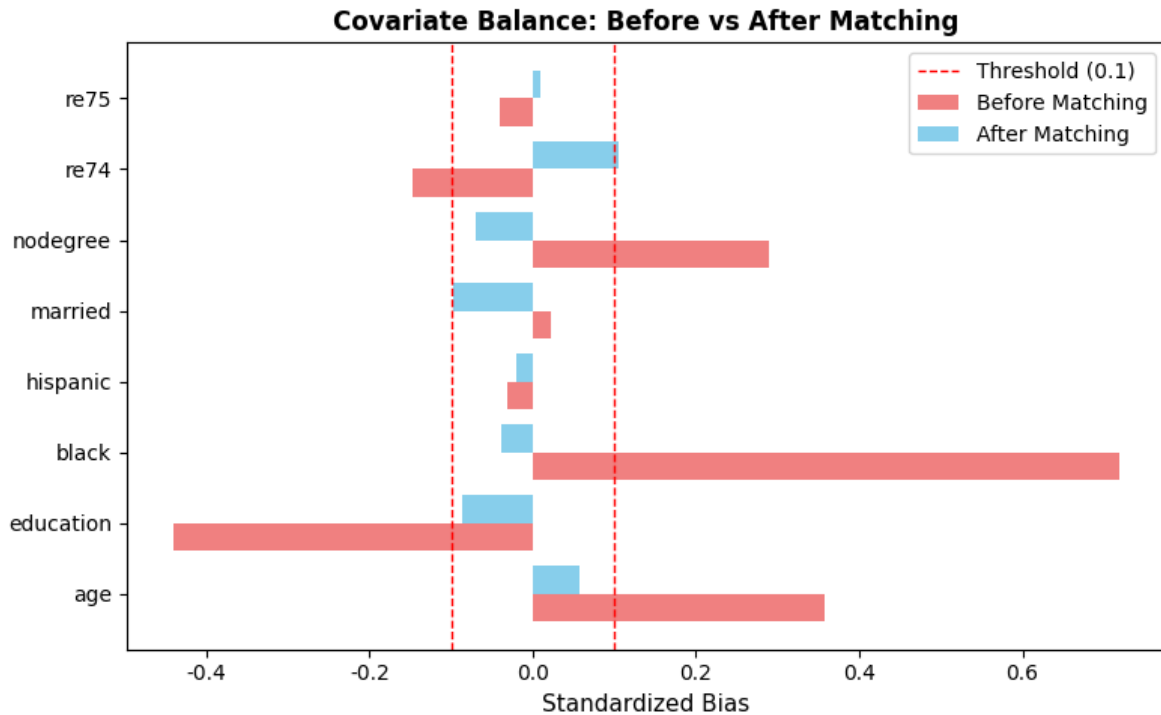
# Love
fig, ax = plt.subplots(figsize=(8, 5))
y_pos = np.arange(len(covariates))
ax.barh(y_pos - 0.2, balance_comparison['Std_Bias'], 0.4,
        label='Before Matching', color='lightcoral')
ax.barh(y_pos + 0.2, balance_comparison['Std_Bias_After'], 0.4,
        label='After Matching', color='skyblue')
ax.set_yticks(y_pos)
ax.set_yticklabels(covariates)
ax.axvline(x=0.1, color='red', linestyle='--', linewidth=1, label='Threshold (0.1)')
ax.axvline(x=-0.1, color='red', linestyle='--', linewidth=1)
ax.set_xlabel('Standardized Bias', fontsize=11)
ax.set_title('Covariate Balance: Before vs After Matching', fontsize=12, fontweight='bold')
ax.legend()
plt.tight_layout()
plt.show()

```

```

:
=====
Variable  Std_Bias  Std_Bias_After
age       0.358     0.057513
education -0.440     -0.086531
black     0.718     -0.038766
hispanic  -0.032     -0.020677
married   0.023     -0.098217
nodegree  0.290     -0.069122
re74     -0.147     0.105057
re75     -0.040     0.008768

```



```
#
naive_est = data[data['treat']==1]['re78'].mean() - data[data['treat']==0]['re78'].mean()

#
matched_est = (matched_data[matched_data['treat']==1]['re78'].mean()
               - matched_data[matched_data['treat']==0]['re78'].mean())

#
X_matched = sm.add_constant(matched_data[['treat']])
model_matched = sm.OLS(matched_data['re78'], X_matched).fit()
reg_est = model_matched.params['treat']

print("      :")
print("==" * 60)
print(f"      :      ${true_effect:,.0f}")
print(f" ( ): ${naive_est:,.0f} ( : {naive_est - true_effect:,.0f})")
print(f"      :      ${matched_est:,.0f} ( : {matched_est - true_effect:,.0f})")
```



```
print(f"      :      ${reg_est:,.0f}  ( : {reg_est - true_effect:,.0f})")
print("=" * 60)
print("\n :      !")
```

```
      :
=====
      :      $1,794
      ( ): $420  ( : -1,374)
      :      $2,448  ( : 654)
      :      $2,448  ( : 654)
=====

      :      !
```

1.

...

2. violation

-
-
-

...

3.

-
-
-

(CBPS)

Imai & Ratkovic (2014)

-
-
-

(Entropy Balancing)

Hainmueller (2012)

-
-
-

...

- - - (Doubly Robust Estimation)

(AIPW)

Augmented Inverse Probability Weighting

$$\hat{\tau}_{AIPW} = \frac{1}{n} \sum_{i=1}^n \left[\hat{\mu}_1(X_i) - \hat{\mu}_0(X_i) + \frac{D_i(Y_i - \hat{\mu}_1(X_i))}{\hat{e}(X_i)} - \frac{(1 - D_i)(Y_i - \hat{\mu}_0(X_i))}{1 - \hat{e}(X_i)} \right]$$

...

- - -

1.
 -
 -
2.
 -
 -
3.
 -
 -
4.
 - < 0.1
 -

- ☐ DAG
- ☐
- ☐ Logit/Probit
- ☐
- ☐
- ☐
- ☐
- ☐

...



- *MatchIt* (R)
- *CausalML* (Python)
- *EconML* (Python)
- *sklearn* (Python)

(Instrumental Variables)

-
-
- (2SLS)
-

...

Z 1. Z D 2. Z D